

Clase 1 Descifrada

La transcripción automática destrozó casi todos los términos técnicos: el reconocimiento de voz oyó español donde el profesor decía inglés. Esto es la clase reconstruida — qué dijo en realidad, qué tienes que entregar, y la tabla para descifrar las próximas.

Lo que tienes que hacer

Sacado de la parte administrativa de la clase (primeros 12 minutos).

11 sept

Actividad 1: instalar el JDK

Mínimo la versión 17. Es el requisito mínimo para trabajar; mencionó una versión más nueva como opcional.

Pronto

Armar equipo de 4 y definir la problemática

Ustedes eligen con quién y qué problema resolver. El proyecto es el mismo todo el semestre.

Pronto

Nombrar jefe de grupo

Solo para avisos y para llevar el control de las entregas parciales.

Cada semana

Entregas: domingo, 11:00 PM

Fue explícito en que sí deja trabajo de fin de semana y no acepta reclamos por eso. Contó el escenario de "se me fue la luz a las 11" como algo que no va a excusar — no dejes la entrega para la última hora.

Por equipo

Una sola persona sube el proyecto

No suban cuatro copias.

+1 punto

Cursos externos opcionales

Abre la próxima semana el de Huawei ICT Academy (inteligencia artificial, básico). Después Red Hat / OpenShift, y en ~15 días unos de Google. Cada uno terminado vale un punto sobre la calificación del examen.

PARTE 1

Las reglas del curso

Lo administrativo, ya desenredado.

El stack base es Java + Spring en el backend, Angular en el frontend, más Insomnia o Postman para probar las APIs. Eso es lo que van a usar todo el semestre.

Puedes cambiar solo una pieza, y el backend no se toca. Si no te gusta Angular, puedes cambiarlo — pero aclaró que React no es un framework sino una biblioteca, así que si lo eliges, sabes que estás tomando algo más chico que trae menos resuelto. El backend se queda en Spring pase lo que pase.

Y ojo: eso aplica al **proyecto**. Para los ejercicios prácticos no hay cambios de stack — todos con lo mismo.

Qué puedes cambiar y qué no



Y esto aplica solo al proyecto — en los ejercicios prácticos todos usan lo mismo.

La regla de "cambias una sola pieza" es lo que hace que el semestre entero se sostenga: si todos tienen el mismo backend, los ejercicios y los criterios de evaluación son comparables.

Todos los exámenes son teórico-prácticos — lo repitió dos veces. No hay examen de solo escribir definiciones.

Los reportes van en una plantilla de LaTeX que está en la plataforma, en la sección de *Recursos*. Ahí mismo hay dos documentos de lectura rápida (uno es de Java) y el enlace a una herramienta en línea para modelar bases de datos. Dijo que va a pedir tarea sobre esas lecturas, así que conviene ir las leyendo.

No todas las entregas llevan reporte escrito: mencionó al menos dos prácticas — la de **acceso a datos** y la de **contenedores** — donde solo quiere el código fuente.

La bibliografía no está en la biblioteca. Lo dijo sin rodeos: "no vas a encontrar nada en la biblioteca, es un hecho". Los libros del temario hay que conseguirlos por otro lado.

Sobre los plazos: las primeras actividades tienen más tiempo del normal porque todavía quedan dos lugares libres en el grupo y podría incorporarse gente tarde. De ahí en adelante, lo normal es **una semana por actividad**.

PARTE 2

La tabla para descifrar

El transcriptor oyó español donde el profesor decía inglés. Estas sustituciones se repiten en toda la grabación y casi seguro van a volver a salir en las próximas — vale la pena tenerlas a la mano.

Hay dos grabaciones de esta misma clase, y cotejarlas sirvió: como el error de reconocimiento es distinto en cada una, lo que una destroza la otra a veces lo deja legible. Así se cerraron cuatro dudas — RBAC, JFreeChart, React Native y DAO pasaron de "probable" a confirmado.

Pero la segunda empieza 16 minutos tarde. Alineando por frases idénticas, arranca en el minuto 16:25 de la primera y termina en el mismo punto. Es decir: cubre toda la teoría, y **nada** de la parte administrativa — que es justo donde estaban los criterios de evaluación.

DICE LA TRANSCRIPCIÓN	DIJO EN REALIDAD	QUÉ ES	CONFIANZA
llave · llama · llamar	Java	El lenguaje. Es la sustitución más frecuente de todas.	seguro
sprint	Spring	El framework de Java para el backend.	seguro
Insomnio	Insomnia	Cliente para probar APIs, alternativa a Postman.	seguro
Postres	Postman	Cliente para probar APIs.	seguro
reacciono	React	La biblioteca de interfaz.	seguro
noir	Node	Aparece en la lista ".NET, Java, PHP, Node".	seguro
jaymen	JMeter	Herramienta de pruebas de carga.	seguro
single · syncreto · single tom	Singleton	El patrón de diseño de una sola instancia. La segunda grabación lo oyó como "single tom", que es la pista más clara.	seguro
d t 2 · d o 2	DTO	Objeto de transferencia de datos.	seguro
dago · dado · "data acceso"	DAO	Objeto de acceso a datos. La segunda grabación lo dejó casi legible: "Data acceso".	seguro
soly	SOLID	Los cinco principios de diseño orientado a objetos.	seguro
vmail	UML	El estándar de diagramas.	seguro
my secret	MySQL	La base de datos.	seguro
pacha cuesta · facha conserva	Apache	El servidor web.	seguro
coche tanto · caballero	Tomcat	El contenedor de servlets de Java.	seguro
jota sp · jspes · foto spr	JSP	JavaServer Pages, la web "tradicional" de Java.	seguro
asp · spx	ASP / ASPX	El equivalente de Microsoft.	seguro

DICE LA TRANSCRIPCIÓN	DIJO EN REALIDAD	QUÉ ES	CONFIANZA
internet información service · internet service	IIS	El servidor web de Microsoft. Necesario para ASP/ASPX.	seguro
a west · la wc	AWS / la nube	Al hablar de pagar disponibilidad a un proveedor. Mezcló AWS y Google en la misma frase.	seguro
monoditos · mono	monolito	El tema principal del día.	seguro
microservicio · vídeos servicios	microservicios	Cuidado: a veces "Microsoft" en la transcripción también es "microservicios". Hay que leer el contexto.	seguro
acabe en coating	Kotlin	Para Android nativo.	seguro
insuit	Swift	Para iOS nativo.	seguro
granolaridad	granularidad	Qué tan fino divides los permisos o los servicios.	seguro
24-65 · 2410365	24/7/365	Disponibilidad continua.	seguro
red hard	Red Hat	De los cursos con punto extra.	seguro
open sheep	OpenShift	La plataforma de contenedores de Red Hat.	seguro
Huawei si academy	Huawei ICT Academy	El curso de IA que da punto extra.	seguro
plantilla de látex · lácteres	plantilla de LaTeX	Para los reportes.	seguro
Cálculo	Angular	Va en la lista del stack base, y abrió la clase diciendo "vamos a trabajar con Angular".	probable
l r · ve a · "el dr. B a c"	RBAC	Control de acceso basado en roles. La segunda grabación lo deletreó como "B a c", lo que cierra la duda.	seguro
chef fisher · jeffery chap	JFreeChart	Biblioteca de gráficas para Java, dicha al hablar de "necesito gráficas". "Jeffery chap" es casi la pronunciación literal.	seguro

DICE LA TRANSCRIPCIÓN	DIJO EN REALIDAD	QUÉ ES	CONFIANZA
briagnation · briack natti	React Native	Lo mencionó como lo que NO quiere para la parte móvil.	seguro
canta faces · pine faces	JSF / PrimeFaces	Componentes de interfaz para Java web.	probable
movies on prime · "what is some background"	on-premise	Servidores propios, en contraste con la nube. Dicho justo antes de la parte de escalabilidad.	probable

PARTE 3

La teoría: arquitectura de software

Aquí empieza la Unidad I. Arrancó a los 13 minutos, después de comentar que se dio cuenta de que el temario anterior no cubría arquitectura de software.

La analogía que usó todo el rato: el arquitecto de una casa. Nadie construye una casa sin planos. Antes de poner un ladrillo hay cimientos, medidas, estructuras, y alguien que supervisa que se sigan. Su punto es que el software es igual — y que la palabra "arquitectura" en software viene literalmente prestada de ahí, porque los primeros que se pusieron a pensar en estructuras, planos y formas de construir cosas fueron los arquitectos de verdad.

"No es llegar y escribir código a la primera. Necesito entender cuál es la naturaleza de la problemática que voy a resolver. Ahora que la entendí, cuál es la mejor estrategia que tengo."

La definición con la que se quedó: **arquitectura es el diseño de más alto nivel de la estructura de un sistema** — un conjunto de patrones y abstracciones que dan un marco claro para implementarlo. No se trata de algoritmos ("no vengo a explicarte el algoritmo de sincronización"), sino de cómo se acomodan las piezas y cómo se relacionan entre sí.

Y advirtió contra el sobrediseño, con un ejemplo que vale la pena recordar porque huele a trampa de proyecto: si tu proyecto es un punto de venta para la tienda de la esquina, no, no necesitas microservicios. "¿Lo requiere? Definitivamente no." Hacer microservicios "porque está de moda" fue justo lo que puso como ejemplo de mala decisión.

Sobre la IA — y esto te va a interesar

Dedicó un buen rato al tema. Su postura no es prohibirla: dijo que **es una herramienta más**. Pero el argumento que dio es exactamente el problema que tú ya identificaste.

"Necesito una clase que implemente el Singleton... y cómo es un Singleton, no tengo ni la menor idea. Pues así como no tenemos ni la menor idea de cómo se construye, así nos va a dar la respuesta esa cosa."

Su punto: si no sabes que un Singleton debe garantizar una sola instancia, que la clase debe ser estática, y que tiene que aguantar mil conexiones simultáneas, **no puedes escribir la especificación** — y sin especificación, lo que te devuelve la herramienta es algo que no sabes evaluar. A la pregunta de a quién va a reemplazar, se contestó solo: al que no se haya enfocado en volverse experto en algo.

Diagramas: cuáles quiere y cuáles no

Van a entregar **diagramas de componentes y de despliegue**. Fue explícito en que *no* son diagramas de casos de uso, y dijo abiertamente que los de secuencia no le gustan — aunque aclaró que es preferencia personal, no una regla.

La idea de componente que usó: **un componente es un cuarto de la casa**. Tú entregas el cuarto; lo que haya adentro (el piso, la ventana, la puerta) puede a su vez ser otro componente. Un componente de autenticación, por dentro, puede tener un MVC, un DTO para decidir qué datos manda, un DAO para el acceso a datos, y un Singleton para no andar abriendo conexiones de más.

Y la analogía de patrón de diseño fue la de costura: bajas de una revista el patrón para hacer un Santa Claus de fieltro, lo sigues al pie de la letra, y te sale. No necesitas saber quién lo diseñó ni por qué funciona. Un patrón de diseño es eso — una solución ya probada y aprobada a un problema que se repite.

Atributos de calidad

Enumeró rendimiento, seguridad, usabilidad, modificabilidad, estabilidad y escalabilidad — y dijo que son difíciles de manejar, que para eso existen los estándares y las normas.

Sobre disponibilidad fue muy escéptico del 100%. Ni los grandes proveedores lo prometen de verdad: siempre te dejan una ventana de error en el contrato. Si contratas la nube de Google y justo te toca su ventana de mantenimiento, te van a enseñar las políticas y te van a decir "yo no te

mentí". La disponibilidad real es tiempo disponible contra tiempo caído, y se mejora con **redundancia y balanceadores de carga** — si se cae un servidor, pasa al siguiente. Salvo que se caigan todos, que también pasa.

Escalar vertical vs. horizontal: vertical es meterle más a la misma máquina; horizontal es agregar más máquinas. Y advirtió que lo horizontal trae complejidad nueva — ahora alguien tiene que repartir el trabajo entre las máquinas y volver a juntar la respuesta. Su imagen: "me pediste un pastel, no me importa quién hizo cada rebanada, yo te tengo que entregar el pastel".

Y sobre probar a escala: contó el caso de un equipo que dijo que su sistema servía para 4600 estudiantes y lo probaron con 10 personas. No puedes ir con los 4600 a pedirles que prueben — **para eso existe JMeter**.

Su conclusión de esta parte, y probablemente lo más examinable del día: el estilo arquitectónico no determina la tecnología. La arquitectura son las reglas y cómo se comporta cada pieza. Que sea .NET, Java, PHP, Node o Python es una decisión posterior.

PARTE 4

El monolito

El tema técnico central de la clase, desde el minuto 48 hasta el final. Corresponde al punto 1.2 del temario.

Qué es: una sola estructura donde viene incluido todo lo que la solución necesita. ¿Necesitas acceso a base de datos? Ahí está. ¿Reportes? Ahí está el reporteador. Todo junto, y al compilar sale **un solo artefacto**: un ejecutable, un instalador, un `.exe` o un `.msi`. El ejemplo que dio fue el instalador de Office de hace años — lo descargabas completo y traía todo dentro.

Adentro puede estar perfectamente organizado en módulos — clientes, proveedores, timbrado de facturas, generación de factura — pero todo se empaqueta y se distribuye como una sola pieza.

A favor

- Fácil de desarrollar
- Fácil de probar
- Pocos puntos de falla
- Es autocontenido: no depende de que otras piezas estén vivas
- Control estricto durante el desarrollo

En contra

- Queda anclado a un stack tecnológico
- Para escalar una parte, hay que escalarlo todo
- Cualquier cambio obliga a regenerar y redespargar la versión completa
- Si algo falla, se cae todo

- El rendimiento puede ser bueno — todo está junto, no hay saltos de red

- En aplicaciones grandes, las malas prácticas se acumulan y se vuelven imposibles de desenredar

El ejemplo de amarre tecnológico que desarrolló: cada tecnología web tradicional arrastra su propio servidor. JSP necesita Tomcat. ASP/ASPX necesita IIS. PHP necesita Apache. Algunas combinaciones se pueden forzar con suficiente configuración, pero no todas — un JSP no corre en un Apache pelón, necesitas sí o sí el contenedor de servlets.

Y el escenario que remata el argumento: tu empresa tiene su sistema monolítico funcionando. La adquiere otra empresa que trabaja en Java, hay que reescribirlo. Después llega alguien que trabaja en Python, hay que reescribirlo otra vez. Eso es la trampa del monolito.

La salida —y aquí es donde arranca el resto del semestre— es dejar de tener la lógica pegada a la pantalla: exponerla como **servicios**. Ya expuesta, la consumes desde donde quieras. La pantalla se encarga solo de la interacción; el peso de la funcionalidad se queda en el backend. Sobre eso después se le cuelga la parte móvil, con la restricción de que la quiere **nativa** — Kotlin para Android, Swift para iOS, nada de React Native ni parecidos.

Ese salto —de un bloque único a lógica expuesta como servicios— es literalmente el puente de la Unidad I a la Unidad II. La clase terminó justo parada en ese borde.

HONESTIDAD

Lo que no se pudo recuperar

Prefiero marcarlo a inventarlo.

Fragmentos ilegibles o dudosos

- Los **criterios de evaluación** (minuto 1:27 a 2:36) están tan destrozados que no se puede reconstruir la ponderación. Se alcanza a ver que habló de examen, proyecto final y ejercicios, pero **los porcentajes no son recuperables** — y la segunda grabación tampoco los rescata, porque empieza después. Esto hay que preguntárselo directamente.
- La **versión del JDK**: "al menos el 17" es claro y se repite. Aparece también un `26` y un `ts` que no logro descifrar con seguridad — podría ser otra versión o una herramienta aparte.
- El segundo documento de lectura rápida: uno es de Java, el otro aparece como `jtv` y no me arriesgo a adivinarlo.
- Los minutos **2:36 a 3:58** (sobre saberes previamente adquiridos y algo de reconciliación de calificaciones) están demasiado corruptos para reconstruirlos con confianza.

- Varios fragmentos sueltos — izzi 11, Williams llama, pop manual — no tienen suficiente contexto alrededor para descifrarlos.